Preorder traversal

//root->left->right

```
#include <bits/stdc++.h>
using namespace std;

//Node structure
class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int x){
        data = x;
        left=right=NULL;
    }
};

void preorder (Node* node,vector<int>& ans){ //preorder traversal
    if(node ==NULL)
        return;
    ans.push_back(node->data);
    preorder(node->left,ans);
    preorder(node->right,ans);
}

int main()
{
    //create the tree and call preorder function
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->right->right = new Node(6);
    vector<int>ans;
    preorder(root,ans);
    for(int x : ans)
        cout<<x <<" ";

    return 0;
}
```

Inorder traversal :-

```cpp
//Inorder traversal
//left->root->right

#include <bits/stdc++.h>
using namespace std;

//Node structure
class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int x){
        data = x;
        left=right=NULL;
    }
};

//inorder -> left->root->right

void inorder(Node* node,vector<int>& ans){

    if(node ==NULL)
        return;
    inorder(node->left,ans);
    ans.push_back(node->data);
    inorder(node->right,ans);

}

int main()
{
    //create the tree and call preorder function
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->right->right = new Node(6);
```

```
    vector<int>ans;
    inorder(root,ans);
    for(int x : ans)
        cout<<x <<" ";

    return 0;
}
```

Postorder traversal :-

```
//Postorder (left->right->root)

#include <bits/stdc++.h>
using namespace std;

//Node structure
class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int x){
        data = x;
        left=right=NULL;
    }
};

//inorder -> left->root->right

void postorder(Node* node,vector<int>& ans){

    if(node ==NULL)
        return;
    postorder(node->left,ans);
    postorder(node->right,ans);
    ans.push_back(node->data);
}

int main()
{
    //create the tree and call preorder function
    Node* root = new Node(1);
```

```
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->right->right = new Node(6);
    vector<int>ans;
    postorder(root,ans);
    for(int x : ans)
        cout<<x <<" ";

    return 0;
}
```

Max Depth of Binary tree :-

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x),
left(left), right(right) {}
 * };
 */
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if(root==NULL)
            return 0;

        int lh = maxDepth(root->left);
        int rh = maxDepth(root->right);

        return 1 + max(lh,rh);
    }
};
```

Diameter of binary tree

```cpp
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x),
left(left), right(right) {}
 * };
 */
class Solution {
public:
    int diameterOfBinaryTree(TreeNode* root) {
        int ans = 0;
        height(root,ans);
        return ans;
    }
private :
    int height(TreeNode* node,int& ans){ //call by reference
        if(node ==NULL)
            return 0;

        int lh = maxDepth(node->left);
        int rh = maxDepth(node->right);

        ans = max(ans,lh+rh);

        return 1 + max(lh,rh);
    }
};
```